

Issues_Summary

Open workable items (current_location = Issues), regenerated from the SQLite single-source-of-truth (§11.4.12).

Counts by Type × Status

Type	Status	Count
Bug	Operator-blocked	1
Bug	Queued	13
Task	In progress	3
Task	Operator-blocked	3
Task	Queued	33
TOTAL		53

Items

ATM ID	Type	Status	Severity	Description
BOB-008	Bug	Operator-blocked	—	RuTracker automated login blocked by CAPTCHA
BOB-065	Task	Queued	High	Lava P2: Egress diagnosis and VPN-hos SOCKS routing (containers pkg/egress)
BOB-066	Task	In progress	High	Lava P3: BOBA_UPSTREAM_PROXY in download-proxy + qBitTorrent-go + Jackett + compose env-forward
BOB-068	Bug	Queued	High	RD2-00: unattributed, unreviewed Auto commit mechanism pushing to main
BOB-069	Task	Queued	High	RD2-40: §11.4.238 QA-discovery-channel ledger — ongoing coverage-escape backfill
BOB-070	Bug	Queued	High	RD2-41: pre-build mutation-marker scan carrier false-positive silently defeats entire pre-build gate
BOB-074	Task	Queued	Medium	RD2-07: DDoS-class testing fully absent from the mandated test-type matrix
BOB-077	Task	Operator-blocked	High	

ATM ID	Type	Status	Severity	Description
				RD2-10: Identify second host running the Auto-commit rsync/sync mechanism (OPERATOR-DECISION)
BOB-078	Task	Queued	High	RD2-11: Once identified, wire Auto-commit mechanism through §11.4.234 dedicated commit/push script OR retire
BOB-079	Task	Queued	Medium	RD2-12: Retroactive attributed history notes for GA-18/21/22/25/26/27 changes (never rewrite published history)
BOB-080	Task	Queued	High	RD2-13: Retroactive Fable-xhigh code review of the substantive Auto-commit diffs
BOB-081	Task	Queued	High	RD2-14: Author CONTINUATION.md Session 15 entry (currently 53 days / 24 commits behind HEAD)
BOB-082	Task	Queued	High	RD2-15: Create BOB-064..067 workable items for the four Lava-porting findings (closes GA-05)
BOB-083	Task	Queued	Medium	RD2-16: Regenerate browser_extension features/codegraph Status.md + Summary/HTML/PDF siblings
BOB-084	Task	Queued	High	RD2-17: Reconcile BOB-008 DB/MD book drift via the workable-items tool
BOB-085	Task	Queued	Medium	RD2-18: Create top-level Boba proxy/merge-service v1.0.0 readiness ledger (closes GA-10)
BOB-086	Task	Queued	Medium	RD2-19: Fix BOB-009/BOB-010 evidence_path + backfill item_history for 56 silent closures
BOB-087	Task	Queued	High	RD2-20: Wire docs_chain / commit-seam sync hook per §11.4.106(F) so DB writes cannot land without MD mirror
BOB-088	Task	Queued	Medium	RD2-21: Complete/verify README Tracked-Items + Status Documents table row-completeness (GA-07 remainder)
BOB-089	Task	Queued	High	RD2-24: RED-first tests for start.sh reload_python/reload_plugins/recreate_stack (closes test-half of GA-27)
BOB-090	Task	Queued	Medium	RD2-25: HelixQA Challenge entry exercising all three start.sh subcommands end-to-end against real compose stack
BOB-092	Bug	Queued	Medium	RD2-27: Remove test_live_stack_evidence.py:265 nnmclu

ATM ID	Type	Status	Severity	Description
				SKIP-on-404 fallback + verify live 200 (closes GA-13)
BOB-093	Task	Queued	High	RD2-28: Live compose bring-up + verify rutracker ReDoS regex bounds deployed to container (closes runtime half of GA-12)
BOB-094	Task	Queued	Medium	RD2-29: Author tests/stress/test_tracker_fetch_stress_chaos.py with §11.4.85 fault injection
BOB-095	Task	Queued	Medium	RD2-30: Author tests/stress/test_scheduler_hooks_sse_stress_chaos.py for Go-side triangle
BOB-096	Task	Queued	Medium	RD2-31: Extend qBitTorrent-go jackett_db_test.go with real process-kill resource-exhaustion fault injection
BOB-097	Task	Queued	Low	RD2-32: Author DDoS-class coverage for exposed download-proxy/merge endpoints (canonical impl of RD2-07)
BOB-098	Task	Queued	Medium	RD2-34: Parametrize 20 hardcoded / Volumes/T7 paths in helixqa banks with PROJECT_ROOT (closes GA-23)
BOB-099	Bug	Queued	Medium	RD2-36: Fix guard-forbidden-commands.sh substring-match false-positive class + add const033-poweroff-signal-triage carrier to EXCLUDE_PATH
BOB-100	Task	Queued	Low	RD2-39: Bump submodules/jackett one commit (canonical impl of RD2-09)
BOB-101	Task	Operator-blocked	High	GA-19/RW-09: Is -profile go parity still a release goal? (gates RW-10..13) — OPERATOR-DECISION
BOB-102	Task	Operator-blocked	Medium	RW-05: LAN-exposure threat model — bind tunnel 127.0.0.1 or keep 0.0.0.0? — OPERATOR-DECISION
BOB-104	Task	In progress	Medium	§11.4.238 followup: CodeGraph 1.5.0 nested-.gitignore regression challenge
BOB-105	Task	Queued	Medium	§11.4.238 followup: mechanical §11.4.227(B) anchor-block-integrity check
BOB-106	Task	Queued	Medium	§11.4.238 followup: §11.4.84 quiescence check helper for the unattributed auto-commit path
BOB-107	Task	Queued	Medium	§11.4.238 followup: pre-dispatch existence check for subagent task-brief source inputs
BOB-109	Task	Queued	Medium	

ATM ID	Type	Status	Severity	Description
				BOB-074 followup: scaling-class test coverage absent from mandated test-type matrix
BOB-110	Task	Queued	Medium	BOB-074 followup: UX-class test coverage (accessibility/usability) absent
BOB-111	Task	In progress	High	BOB-074 followup: configure real rate limiting for boba's 3 public HTTP endpoints
BOB-113	Task	Queued	Low	BOB-074 followup: add wrk to dev tooling for DDoS/load challenges
BOB-114	Task	Queued	Medium	BOB-074 followup: self-validation golden bad fixture for the rate-limit detector
BOB-118	Bug	Queued	High	README.md python-tests badge claims 585 passing; pytest -collect-only measures 5235 (9x stale/wrong)
BOB-120	Bug	Queued	Critical	3rd forced-logout incident 2026-08-18 23:45:49 — SIGKILL user@1000 + preventive-timer-inside-user-slice architectural gap
BOB-121	Task	Queued	Important	External watchdog for the forced-logout architectural gap (task #85, incident #3)
BOB-129	Bug	Queued	Medium	Potential production slowapi/starlette defect flagged by Task 105 subagent
BOB-131	Bug	Queued	Medium	qbittorrent-proxy podman common crash — pre-existing, surfaced during BOB-12 investigation
BOB-135	Bug	Queued	Low	Test isolation: test_list_hooks_after_create fails in bulk suite (Permission denied /config)
				WHAT. A single 2026-08-20 sweep found FOUR workable-item rows whose tracked state contradicted reality: BOB-076 (closed by commit 99a486e on 2026-08-15, whose message literally says "closes BOB-076", still Queued), BOB-1 (fix landed AND covered by a permanent guard assertion, still Queued), BOB-091 (implemented under d1a8479 / a3b16bc 2a0b543, still Queued), and BOB-008 whose Evidence field still quoted a diagnostic string that BOB-117 removed from source. CORRECTION 2026-08-20 (11.4.6). An earlier revision of this item claimed the enabling defect was that "workable-items diff is blind to body_md drift". That was WRONG and is corrected here rather than quietly edited. The
BOB-136	Task	Queued	High	

ATM ID	Type	Status	Severity	Description
				engine is NOT blind. The FLAGLESS INVOCATION is: workable-items diff -d docs/workable_items.db -> "diff: DB and Markdown are in sync" (opens ZERO .m files) workable-items diff -db docs/workable_items.db -issues docs/Issues.md -fixed docs/Fixed.md -> "~ BOB-008 body differs (md=1346 bytes db=1333 bytes)" -> "diff: 1 difference(s) Both measured on the same seeded body only drift. An independent agent proved the mechanism with strace: the flagless form never opens any Markdown file. So this is a 11.4.201(6) FALSE-NULL of the CALLER, not a gap in the comparison logic. Importantly, pre_build_verification.sh invariant 17 already passes both flags, so the STANDING GATE was never blind — the drift above was not caused by a blind gate. THE REAL DEFECTS, restated honestly. (1) The flagless form is a false null trap: it reports a confident, reassuring "in sync" while checking nothing. Any caller using it is a green-reporting blind check. It should either default to the conventional Issues.md/Fixed.md paths or REFUSE with a clear error, never return a clean verdict for a comparison it did not perform. (2) Work commonly lands under commit messages that do not carry the ATM id (GA-14/15/16 for BOB-091), so nothing mechanically links a commit back to its row, and closure depends on a human remembering. 11.4.227: an anchor done state is its SEAM landing, not its TEXT landing. ACCEPTANCE. (a) The flagless diff invocation no longer returns a clear verdict without reading Markdown, with a paired 1.1 mutation proving it. (b) A repo-wide grep confirms no caller uses the flagless form. (c) A sweep that flags any non-terminal item whose id appears in a merged commit message, so done-but-open rows are found mechanically rather than by a human noticing. (d) Honest boundary: this does not claim to prevent all drift. RELATED. scripts/hook docs-sync-commit-seam.sh (added under

ATM ID	Type	Status	Severity	Description
				BOB-087) now enforces doc/DB sync at the commit seam and passes the flags explicitly, plus an independent body_md re-parse oracle. EVIDENCE. docs/qa/BOB-117/closure-evidence.md, docs/qa/BOB-076/closure-evidence.md, docs/qa/BOB-091/closure-evidence.md. Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T14:46:52Z Reported-By: Claude What (the report verbatim): The merge service on port 7187 wedges after hours of uptime: the port stays bound and the container keeps reporting “healthy”, but every HTTP request hangs until the client times out. Measured 2026-08-20 16:41 UTC+2 on container up 3h53m: curl http://localhost:7186/ -> HTTP 200 in 0.096s (proxy, same process) curl http://localhost:7187/ -> HTTP 000 after 6.0s (merge service, WEDGED) Both ports are served by the SAME process (pid 2330069, fd 3 = 7186, fd 7 = 7187), so this is not a crashed worker – one loop inside a live process has stopped servicing requests while another in the same process is fine. Thread state at the time of the wedge (/proc/2330069/task, 16 threads): - tid 2330529: state R, wchan 0 <- ONE thread spinning on CPU - tid 2330528: state S, wchan do_sys_poll <- the healthy 7186 poll loop - the other 14: state S, wchan futex_wait_queue This is the GIL-starvation signature: a thread busy-looping in Python holds the GIL and every other thread queues on the GIL futex. 7186 survives because do_sys_poll releases the GIL; the 7187 async loop needs sustained GIL time to service a request and starves. Process CPU was 20.6% while idle. Corroborating evidence: seven sockets held by the process sit in CLOSE-WAIT with unread bytes still in the receive queue (Recv-Q 162, 162, 162, 79, 1, 1, 1) – clients sent request and hung up, and the app never read it or closed the fd. The listener had also accumulated an unaccepted backlog (Recv-Q 6 on the 7187 LISTEN socket) at first observation. The container log’s last
BOB-137	Bug	Queued	High	

ATM ID	Type	Status	Severity	Description
				line is 2h before the observation, so the service processed nothing in that window. NOT fd exhaustion: only 48 of 16384 fds were open. ROOT CAUSE NOT ESTABLISHED. All four while True loops in the service (routes.py:166, search.py:1169, streaming.py:161, streaming.py:359) are correctly bounded and awaited, so the spin is not a naive unslept loop. py-spy could not attach to name the spinning frame – the host has kernel.yama.pttrace_scope=1, which denies non-child attach. Per the §11.4.1 Iron Law no fix is proposed until the spinning frame is identified. Next diagnostic step: obtain a Python stack. Either run py-spy INSIDE the container (musl wheel), or set pttrace_scope=0 for the duration of one dump, or add a SIGQUIT/faulthandler.register() dump hook to main.py so the running service can be asked for its own stacks without pttrace. DISCOVERY CHANNEL (§11.4.238): found by an agent probing the host by hand while investigating an unrelated test-suite result – NOT by automated QA. This is a coverage escape in its own right; see the sibling item on the health check that was structurally incapable of observing it. Affected scope / file-scope manifest: download-proxy/src/main.py, download-proxy/src/merge_service/, download-proxy/src/api_streaming.py, docker-compose.yml (qbittorrent-proxy) Reproduction / context: Leave the qbittorrent-proxy container running for several hours with search traffic (a full tests/security run is sufficient). Then: curl -max-time 6 http://localhost:7186/ returns 200; curl -max-time 6 http://localhost:7187/ returns 00. Confirm with: ss -ltnp grep 7187 (unaccepted backlog) and awk '{print \$3}' /proc//task/*/stat (one R thread, re_futex_wait_queue). Acceptance criteria The spinning frame is identified from a real Python stack dump (not inferred), the busy-loop is fixed at its root, and a regression guard proves 7187 still answers after a sustained-traffic soak.

ATM ID	Type	Status	Severity	Description
BOB-139	Bug	Queued	Medium	Evidence: before/after curl timings on both ports plus a thread-state census showing no permanently-R thread. Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T14:47:35Z Reported-By: Claude What (the report verbatim): Both SSE generators in download-proxy/src/api/streaming.py (lines ~153 and ~351) guard their while True loop with this helper: <code>async def _client_gone() -> bool: if request is None: return False try: return await request.is_disconnected() except Exception: return False</code> The bare <code>except Exception: return False</code> means: if the disconnect check itself ever raises, the generator concludes the client is STILL CONNECTED and keeps streaming - forever. That is fail-OPEN on the only condition that terminates the loop, in a code path that holds a socket and a task. This is the §11.4.252 fail-closed rule inverted: the safe default for “I cannot determine whether the client is gone” is to treat it as gone and stop streaming (the client can always reconnect - SSE is designed for that), not to keep an unbounded stream alive on an unresolvable signal. It is also a §11.4.201(6) false-null: a raising probe and a genuinely-connected client return the identical False, so the loop cannot distinguish “client is here” from “I am blind”. Corroborating observation (2026-08-20): seven sockets held by the merge-service process were sitting in CLOSE-WAIT with unread request bytes, clients had hung up and the server had not noticed. That is consistent with (though not yet proven to be caused by) this fail-open, and it is the reason this is filed rather than left as a style note. HONEST BOUNDARY: not proven to be the cause of the 7187 wedge - both loops DO call <code>await asyncio.sleep(poll_interval)</code>, so they yield and would not busy-spin. This is filed as a real defect on its own merits (an unbounded stream on an unresolvable signal, plus a leaked socket and task), not

ATM ID	Type	Status	Severity	Description
BOB-141	Task	Queued	Low	as the wedge's root cause. Affected scope / file-scope manifest: download proxy/src/api/streaming.py (_client_gon at ~line 153 and ~line 351) Reproduction / context: Monkeypatch Request.is_disconnected to raise, open SSE stream, disconnect the client, and observe the generator never terminates (the loop keeps yielding and the task is never reclaimed). Acceptance criteria A raising disconnect probe terminates t stream (fail-closed per §11.4.252) rather than continuing it, AND a normally-connected client still streams uninterrupted (§11.4.201 both directions). Guard: a unit test for each SSE generator covering raise -> terminate and connected -> continue. Reported-Via: §11.4.202 reporting directive task on 2026-08-20T14:56:38Z Reported-By: Claude What (the report verbatim): CLAUDE.md's Architecture section states: "qbittorrent-proxy-go (Gin, opt-in via -profile go) - replaces the Python proxy on 7186, 7187, 7188" The container cannot deliver that. Read from source rather than prose: qBitTorrent-g Dockerfile:16 CMD ["/app/qbittorrent-proxy"] (ONE binary) qBitTorrent-go/ Dockerfile:15 EXPOSE 7187 7188 (declares 2) cmd/qbittorrent-proxy/main.go r.Run(fmt.Sprintf(":%d", cfg.ServerPort)) (binds 1) internal/config.go:58 ServerPort = MERGE_SERVICE_PORT (default 7187) So the qbittorrent-proxy-go container binds 7187 ONLY. webui-bridge is a SEPARATE binary (cmd/webui-bridge, /bridge/health, cfg.BridgePort) that this container never starts, and nothing in it binds 7186 either - although the component service does set PROXY_PORT=7186 and BRIDGE_PORT=7188 in its environment which reinforces the wrong impression. EXPOSE likewise declares a port nothing binds. WHY THIS MATTERS BEYOND TIDINESS: this prose was used as the source of truth when first authoring config/served_ports.yaml, producing a manifest entry of [7186, 7187, 7188] for

ATM ID	Type	Status	Severity	Description
BOB-143	Bug	Queued	Medium	that service. The healthcheck gate built on it then FAILED a service whose healthcheck was already correct - a §11.4.201(1) false-positive refusal caused directly by trusting the doc over the Dockerfile. The manifest was corrected against source; the doc was not, and will mislead the next reader the same way. Either the doc is wrong, or the Go container is under-provisioned relative to intent (it should also run webui-bridge and a 7186 listener). Determining WHICH is part of this task - do not simply reword the doc to match the current binary if the intended design was a three-port container. Related: the Go service's compose block sets PROXY_PORT and BRIDGE_PORT that no process in the container consumes, and EXPOSE lists 7188 unbound. Whichever way the above resolves, those should agree with reality afterwards. Affected scope / file-scope manifest: CLAUDE.md (Architecture + Port Map), docker-compose.yml (qbittorrent-proxy-go env/EXPOSE), qBitTorrent-go/Dockerfile Reproduction / context: Read qBitTorrent-go/Dockerfile:16 (single CMD) against CLAUDE.md's 'replaces the Python proxy on 7186, 7187, 7188'; confirm cfg.ServerPort resolves to MERGE_SERVICE_PORT=7187 in internal/config/config.go:58. Acceptance criteria: Doc and container agree, with the direction of the fix decided deliberately (correct the doc, or provision the container to match the documented intent). PROXY_PORT/BRIDGE_PORT env and EXPOSE lines agree with what the container actually binds. Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T15:08:18Z Reported-By: Claude What (the report verbatim): .worktrees/ci-split-workflows/ (13M) and .worktrees/completion-initiative-phase-0/ (33M) are ORPHANED: git worktree list reports only the main checkout, so neither is a registered worktree. Each contains a .git POINTER FILE whose target gitdir no

ATM ID	Type	Status	Severity	Description
				longer exists, so git cannot resolve HEAD, branch, or status inside them - every query returns empty. They are gitignored (.gitignore:122), so nothing tracks them and nothing will ever notice them drifting. WHY THIS IS NOT COSMETIC they pollute the scan scope of whole-tree gates and manufacture false findings. Measured 2026-08-20 by the §11.4.32 sweep: - cm_test_mock_pid_explicit_int reported 2 violations at tests/unit/merge_service/test_deadline_tunable.py:44 - BOTH inside these orphaned trees. The MAIN tree's copy of that file is already hardened (it sets mock.pid = 12345 and patches os.killpg/os.getpgid) and passes the gate cleanly. The finding read as a live §11.4.263 / BOB-126-class defect and was not one. - 6 of the 57 "missing anchor carrier" files flagged by the propagation gates were likewise .worktrees/**. That the §11.4.201(1) false-positive shape sourced from scan scope, and it costs re-investigation time: a reader triaging "2 live BOB-126 violations" reasonably treats it as a host-safety emergency. CLARIFICATION (established during triage, so the record is not alarming): these trees are NOT a kill(-1) vector. Their download-proxy/src/merge_service/search.py contains ZERO os.killpg calls they predate that cleanup code entirely so there is no unguarded signal call to reach. Additionally pyproject.toml sets testpaths = ["tests"], so a plain pytest run does not collect from .worktrees/. No host-safety risk was found. The defect is scan-scope noise plus 46M of unreferenced disk. WHY REMOVAL IS NOT DONE AUTONOMOUSLY (§11.4.122 / §11.4.124 / §11.4.101): because git cannot resolve their HEAD, is NOT possible to prove their contents are merged into main. Deleting unprovable-provenance work is exactly the irreversible, operator-owned decision §11.4.122 reserves. Two options for the operator: (a) confirm removal (they are stale dev scratch dirs) - reversible only

ATM ID	Type	Status	Severity	Description
BOB-144	Bug	Queued	Low	from backup, so a §9.2 pre-op backup should precede it; or (b) keep them and add .worktrees/ to the gate scan-scope exclusion list as a §11.4.224(E)-fenced, checked-in, justified entry. Either way the exclusion list is the cheaper immediate mitigation and does not destroy anything Affected scope / file-scope manifest: .worktrees/ci-split-workflows/, .worktrees/completion-initiative-phase-0/, gate scan-scope config Reproduction / context: git worktree list shows only the main checkout; git -C .worktrees/ Reported-Via: §11.4.202 reporting directive bug on 2026-08-20T15:53:57Z Reported-By: Claude What (the report verbatim): /theme/stream in download-proxy/src/api/routes.py:167 calls the disconnect probe with NO guard at all: while True: if await request.is_disconnected(): break If that probe raises, the generator dies with an UNCAUGHT exception. IMPORTANT — this is NOT the BOB-139 fail-open. The effect here is fail-CLOSED: the stream stops, and the enclosing finally: store.unsubscribe(queue) still runs, so the subscriber queue is released and nothing leaks. The outcome is CORRECT; the manner is not. What is wrong with it: - it terminates via an uncaught traceback rather than a clean SSE close event, so the client sees a truncated stream instead of a reason; - the failure is not logged as a probe failure, so a systematically raising probe would show up as recurring tracebacks with no diagnosis; - it is inconsistent with the two SSE generators in streaming.py, which after BOB-139 emit event: close with reason disconnect_probe_failed and log a warning. Three call sites of the same probe now behave two different ways. The BOB-139 fix deliberately did not touch this file (ownership boundary, §11.4.119), and flagged it honestly rather than fixing it out of scope. Acceptance: theme/stream uses the same probe-failure discipline as the streaming.py

ATM ID	Type	Status	Severity	Description
				generators — a clean close event with t disconnect_probe_failed reason plus a logged warning — proven in BOTH directions (§11.4.201(1)): a raising prob closes the stream cleanly AND a normally-connected client still streams uninterrupted. Prefer reusing the share helper introduced by BOB-139 rather than a third copy of the logic (§11.4.251 Honest boundary (§11.4.6): the production probe-failure rate is UNKNOWN — nobody has measured ho often is <code>is_disconnected()</code> actually raises. This is filed on the inconsistency and th missing diagnosis, not on a measured incident rate. Affected scope / file- scope manifest: download-proxy/src/a routes.py (~line 167, <code>stream_theme</code>) Reproduction / context: Monkeypatch <code>Request.is_disconnected</code> to raise, open <code>theme/stream</code>, observe the generator di with an uncaught traceback rather than emitting event: close with reason <code>disconnect_probe_failed</code> as <code>streaming.p</code> now does. Acceptance criteria: Same probe-failure discipline as <code>streaming.py</code> (clean close + <code>disconnect_probe_failed</code> reason + logged warning), verified both directions, reusing the BOB-139 helper rather than a third copy.